

Azure Production Deployment — Detailed Reference

This document explains how the platform is deployed and operated in the Azure production environment (`video-extract-prod`): what Azure services are used, what role each plays, how the services talk to each other, how scalability is achieved, and how new versions reach production via the CI/CD pipeline.

1. Production Resource Group

All production resources live in a single Azure Resource Group:

```
video-extract-prod (location: configurable via Terraform variable)
```

Everything in this document is inside that resource group unless stated otherwise. Terraform state for this environment is stored remotely in a separate, shared storage account (`tfstatevideoextract`), resource group (`terraform-state-rg`) so it survives any accidental deletion of the application resource group.

Infrastructure is fully defined as Terraform code in `infrastructure/terraform/envs/prod/main.tf` and the modules it calls. Running `terraform apply` from that directory is the complete, repeatable definition of the production environment.

2. Azure Services and Their Roles

2.1 Azure Container Registry (ACR)

Terraform module: `modules/acr/` — SKU: Standard

ACR is the private Docker image registry for the platform. All eight application images are built by GitLab CI and pushed here before any deployment happens.

What it stores	How it is used
<code>api-gateway:<sha></code>	Pulled by ACA on container app update
<code>agent-orchestrator:<sha></code>	Same
<code>preprocessing-worker:<sha></code>	Same
<code>notification-worker:<sha></code>	Same
<code>mcp-server-analysis:<sha></code>	Same
<code>mcp-server-processing:<sha></code>	Same
<code>angular-shell:<sha></code>	Same
<code>librechat:<sha></code>	Same

Images are tagged with the git commit SHA (`CI_COMMIT_SHORT_SHA`) on every build, and tagged `:latest` only after tests pass on the `main` branch. This means every running container in production can be traced back to an exact git commit.

ACA pulls images from ACR using admin credentials (`acr-password`) stored as a container app secret inside each container app definition.

2.2 Azure Container Apps (ACA) + Log Analytics

Terraform module: `modules/aca/`

ACA is where all application containers run. A single **Container Apps Environment** named `videoextract-prod-cae` hosts all nine container apps. The environment is backed by a **Log Analytics Workspace** (`videoextract-prod-law` , PerGB2018 SKU, 30-day retention) that aggregates stdout/stderr from every container.

The nine container apps and their configuration:

Container App	Port	Ingress	CPU	Memory	Scale trigger
postgresql	5432	Internal TCP	0.5 vCPU	1 GiB	Fixed: min=1, max=1
api-gateway	8000	External HTTP	0.5 vCPU	1 GiB	HTTP concurrency $\geq$ 50
agent-orchestrator	8001	Internal HTTP	1.0 vCPU	2 GiB	Service Bus queue depth
mcp-server-analysis	8100	Internal HTTP	0.5 vCPU	1 GiB	HTTP concurrency $\geq$ 20
mcp-server-processing	8200	Internal HTTP	1.0 vCPU	2 GiB	Service Bus queue depth
preprocessing-worker	—	None	1.0 vCPU	2 GiB	Service Bus queue depth
notification-worker	—	None	0.25 vCPU	0.5 GiB	Service Bus queue depth
angular-shell	80	External HTTP	0.25 vCPU	0.5 GiB	HTTP concurrency $\geq$ 50
librechat	3080	External HTTP	0.5 vCPU	1 GiB	HTTP concurrency $\geq$ 50

Three container apps are publicly reachable (`external_enabled = true`): `angular-shell` , `librechat` , and `api-gateway` . All others are internal — reachable only from within the ACA environment by their service name (e.g. `http://agent-orchestrator` , `http://mcp-server-analysis`).

In production, `min_replicas = 1` and `max_replicas = 20` for all application containers. This keeps the platform warm (no cold-start latency for users) while allowing up to 20 instances of any service under load. PostgreSQL is always fixed at exactly one replica because it is stateful.

2.3 Azure Blob Storage

Terraform module: `modules/storage/` — replication: ZRS (Zone-Redundant Storage in prod)

Blob Storage is the single storage layer for all media. Every video file — raw uploads, extracted keyframes, intermediate segments, and final output — lives here.

Container layout:

```
videos/  
  <user_id>/  
    original/      raw uploaded video files  
    keyframes/     1fps frame images extracted by preprocessing-worker  
    segments/      intermediate clips created during processing  
    processed/     transformed clips (speed, color, resize)  
    outputs/       final compiled output videos  
  
assets/  
  <session_id>/  
    <uuid>/        non-video uploaded files (JSON, CSV, TXT, images)
```

How each service uses it:

Service	Access pattern
Browser (Angular)	Direct <code>PUT</code> to Blob using a short-lived SAS write token from the API gateway
preprocessing-worker	Reads the raw upload; writes keyframe images; writes keyframe index JSON
mcp-server-analysis	Reads keyframe images to run analysis tools
mcp-server-processing	Reads segments; writes processed clips and the final output video
agent-orchestrator	Writes the final output blob URL to PostgreSQL
api-gateway	Generates SAS tokens for upload; generates signed Front Door URLs for download

Zone-redundant replication means data is replicated synchronously across three availability zones within the region. Blob versioning is enabled with a 7-day soft-delete policy for accidental deletion protection.

The same storage account also hosts an **Azure Files share** (`postgres-data` , 128 GB quota in prod) that is mounted into the PostgreSQL container as a persistent volume. This is how the database survives container restarts without using a managed database service.

2.4 Azure Service Bus

Defined in: `envs/prod/main.tf` — SKU: Premium (capacity 1)

Service Bus is the asynchronous backbone of the platform. Every stage of the video processing pipeline communicates via Service Bus queues rather than direct synchronous calls. This decouples services, enables retries, and allows each stage to scale independently.

Five queues:

Queue	Published by	Consumed by	Triggered when
<code>video-uploaded</code>	Azure Blob Storage (event trigger)	<code>preprocessing-worker</code>	User completes a video upload
<code>video-indexed</code>	<code>preprocessing-worker</code>	<code>agent-orchestrator</code> (SB consumer)	Keyframe extraction is complete
<code>job-queued</code>	<code>api-gateway</code>	<code>agent-orchestrator</code> (SB consumer)	User submits a processing job
<code>job-completed</code>	<code>agent-orchestrator</code>	<code>notification-worker</code>	Agent finishes successfully
<code>job-failed</code>	<code>agent-orchestrator</code>	<code>notification-worker</code>	Agent encounters an unrecoverable error

Each queue has `max_delivery_count = 10` — if a consumer fails to process a message 10 times, the message is dead-lettered. This prevents poison messages from blocking the queue indefinitely while still allowing transient failures to be retried.

The Premium SKU is used in production because it provides dedicated capacity (no resource sharing with other tenants) and supports virtual network integration if network isolation is added later.

2.5 Azure Front Door

Terraform module: `modules/frontdoor/` — SKU: Standard

Front Door sits between the internet and the platform. It provides:

- TLS termination and HTTPS enforcement** — all HTTP traffic is redirected to HTTPS at the Front Door edge; traffic to the ACA `api-gateway` origin is always HTTPS.
- CDN caching** — static and frequently-accessed output files are cached at Front Door edge nodes globally, reducing latency for geographically distributed users.
- WAF-ready entry point** — the Standard SKU can have a Web Application Firewall policy attached without changing the application.
- Signed URL generation for output delivery** — the `api-gateway` and `notification-worker` generate HMAC-SHA256 signed Front Door URLs for output video downloads. The `OUTPUT_URL_MODE=frontdoor` environment variable on the `api-gateway` activates this path. Signed URLs are time-limited, so output videos are not permanently accessible via a guessable URL.

Front Door is configured with a single origin group pointing to the `api-gateway`'s ACA FQDN. All traffic (`/*`) is routed through it.

The Front Door endpoint hostname is injected into both the `api-gateway` container (`FRONT_DOOR_ENDPOINT`) and the `notification-worker` container (`FRONT_DOOR_HOSTNAME`) so both can construct signed URLs without hardcoding the hostname.

2.6 Azure PostgreSQL (container-based)

PostgreSQL 15 runs as an ACA container app (`postgres:15-alpine`) within the same Container Apps Environment as the application services. It is **not** a managed Azure Database for PostgreSQL service — it is a self-managed container backed by an Azure Files volume for persistence.

Why a container rather than a managed service? Cost and simplicity at this project scale. A managed PostgreSQL Flexible Server would add significant monthly cost. The Azure Files volume provides durability equivalent to a managed service for the data volumes this platform needs.

Configuration decisions:

- Fixed at exactly one replica (`min_replicas = 1` , `max_replicas = 1`) — PostgreSQL is stateful and cannot horizontally scale without read replicas or Citus, neither of which is needed here.
- `PGDATA` is set to `/var/lib/postgresql/data/pgdata` (a subdirectory) because Azure Files mounts a `lost+found` directory at the volume root, which prevents PostgreSQL's `initdb` from running if `PGDATA` points directly to the mount.
- Internal TCP ingress only — no external access; all database-consuming services connect via the internal hostname `postgresql:5432`.
- No automated backups. Azure Files point-in-time restore or manual snapshots are the backup mechanism if needed.

Tables stored:

Table	Purpose
users	User accounts and emails
sessions	Groups of uploads and jobs
videos	Uploaded video metadata
video_keyframe_index	Per-frame URLs from the preprocessing worker
session_assets	Unified blob index per session (videos + files + outputs)
assets	Non-video uploaded files
jobs	Job records with status, prompt, video references
job_steps	Per-step agent execution log
outputs	Output video references
app_settings	Runtime configuration (e.g. tool_frontier_model)

2.7 Azure Key Vault

Terraform module: modules/keyvault/ — ve-prod-kv , Standard SKU, purge protection enabled

Key Vault stores all sensitive values for the production environment:

Secret name	Value
anthropic-api-key	Anthropic API key used by agent-orchestrator and mcp-server-analysis
db-password	PostgreSQL admin password
storage-connection-string	Blob Storage connection string
servicebus-connection-string	Service Bus connection string
acs-connection-string	Azure Communication Services connection string
appinsights-connection-string	Application Insights connection string

Purge protection is enabled in production, meaning secrets cannot be permanently deleted for the soft-delete retention period (7 days) even by the vault owner. This protects against accidental or malicious secret destruction.

Current wiring (Phase A): Secrets are populated by Terraform and currently injected into ACA containers as plain environment variables via Terraform env blocks. The values flow: Terraform variable → Key Vault secret (stored) + ACA container env var (injected at deploy time).

Planned wiring (Phase B): ACA managed identities will be granted Key Vault Secrets User role, and container apps will reference secrets via key_vault_secret_id rather than embedding values at deploy time. This means secrets will be fetched at container startup from Key Vault directly, and rotating a secret will require only a container restart rather than a Terraform re-apply.

2.8 Azure Communication Services (ACS)

Terraform module: modules/appcommunication/ — videoextract-prod-accs

ACS sends transactional emails when a job completes or fails. The notification-worker receives a JOB_COMPLETED or JOB_FAILED Service Bus message, queries PostgreSQL for the user's email address, generates a signed Front Door download URL, and calls ACS to send the email.

Email content includes:

- The original prompt text
- A time-limited signed download link for the output video (or failure reason)
- Processing duration

ACS is set to NOTIFICATION_MODE=acss in production. In local development this is stdout (emails are logged instead of sent). After Terraform creates the ACS resource, a sender email domain must be manually verified in the Azure portal before emails can be delivered. This one-time step is not automatable via Terraform.

2.9 Azure Entra External ID

Entra External ID is not provisioned by Terraform — it is a tenant-level resource that is created once manually. The Terraform code accepts entra_tenant_id and entra_client_id as variables and threads them into the api-gateway container

environment.
Role in the platform:

1. User enters their email in the Angular shell.
 2. Angular redirects to Entra External ID, which sends a magic link email.
 3. User clicks the link; Entra issues a signed JWT.
 4. Angular attaches the JWT as a Bearer token to every API request.
 5. The `api-gateway` validates the JWT on every request against the Entra JWKS endpoint (`https://login.microsoftonline.com/<tenant_id>/discovery/v2.0/keys`). This validation is implemented in `backend/api-gateway/src/middleware/auth.ts` .
- No password storage, no session management, and no auth implementation in the application — Entra handles the full identity lifecycle.

2.10 Azure Application Insights

Terraform module: `modules/appinsights/` — `videoextract-prod-ai` , `web` type
Application Insights provides full observability for the production environment with **zero custom instrumentation code** — only auto-instrumentation packages are used.

Service type	Package	Initialization
Node.js (api-gateway)	<code>applicationinsights</code> npm package	<code>appInsights.setup(...).start()</code> before all other imports in <code>src/index.ts</code>
Python services	<code>azure-monitor-opentelemetry</code>	<code>configure_azure_monitor()</code> before FastAPI app creation in each <code>app/main.py</code>

All services receive `APPLICATIONINSIGHTS_CONNECTION_STRING` as an environment variable. When the variable is absent (local development), both SDKs are a no-op — no code change is needed.

Auto-instrumentation captures:

- **Distributed traces** across all services (HTTP dependencies are automatically correlated via trace context propagation)
- **Dependency maps** showing which services call which
- **Request rates, failure rates, and response times** per endpoint
- **Exception tracking** with stack traces

Application Insights is linked to the same Log Analytics workspace as the ACA environment (`videoextract-prod-law`), so container stdout logs and APM telemetry are queryable together in the Azure portal.

3. How the Services Connect

3.1 Network topology

All ACA containers live in the same Container Apps Environment (`videoextract-prod-cae`). Within that environment, services address each other by container app name over internal HTTP:

<code>api-gateway</code>	→ <code>http://agent-orchestrator</code>	(port 8001, POST <code>/run</code>)
<code>agent-orch.</code>	→ <code>http://mcp-server-analysis</code>	(port 8100, SSE tool calls)
<code>agent-orch.</code>	→ <code>http://mcp-server-processing</code>	(port 8200, SSE tool calls)
<code>all services</code>	→ <code>postgresql</code>	(port 5432, TCP)

No service ports are exposed externally except `angular-shell` (port 80), `librechat` (port 3080), and `api-gateway` (port 8000). Those three are accessible via their ACA-assigned FQDNs and routed through Azure Front Door.

3.2 Data flow — upload

```
Browser
  POST /v1/sessions → api-gateway          (creates session record in PostgreSQL)
  POST /v1/videos   → api-gateway          (generates SAS write token for Blob)
  PUT <SAS URL>     → Azure Blob Storage    (browser uploads directly; no server egress)
  Blob Storage      → Service Bus: video-uploaded
  preprocessing-worker ← Service Bus
    → downloads video from Blob
    → extracts keyframes with FFmpeg
    → uploads keyframe images to Blob
    → writes video_keyframe_index rows to PostgreSQL
    → publishes Service Bus: video-indexed
```

Browser uploads go directly to Blob Storage using a short-lived SAS write token. The api-gateway never proxies video bytes in production — this eliminates server egress cost for uploads, which would be significant for multi-gigabyte video files.

3.3 Data flow — job execution

```
LibreChat iframe
  POST /v1/chat → api-gateway          (X-Session-Id header forwarded)
  api-gateway → POST /run → agent-orchestrator (synchronous path for chat)
OR
  POST /v1/jobs → api-gateway
    → PostgreSQL: jobs (status=queued)
    → Service Bus: job-queued

agent-orchestrator (SB consumer picks up job-queued)
  → reads keyframe_index from PostgreSQL
  → CrewAI crew.kickoff():
    Planner Agent → interprets prompt → extraction plan (LiteLLM → Anthropic API)
    Analysis Agent → calls mcp-server-analysis tools via SSE
    Processor Agent → calls mcp-server-processing tools via SSE
  → output video written to Blob Storage
  → PostgreSQL: jobs (status=completed, output_url=...)
  → Service Bus: job-completed

GET /v1/jobs/{id}/stream → api-gateway
  → SSE stream polls PostgreSQL → pushes status events to Angular

notification-worker (picks up job-completed)
  → fetches user email from PostgreSQL
  → generates signed Front Door URL for output video
  → sends email via Azure Communication Services
```

3.4 Data flow — output delivery

```
Angular
  GET /v1/outputs/{id} → api-gateway
    → generateSignedDownloadUrl() in blobService.ts
    → HMAC-SHA256 signed Front Door URL (OUTPUT_URL_MODE=frontdoor)
    → returns signed URL to browser

Browser
  GET <signed Front Door URL>
    → Front Door validates signature
    → Front Door fetches from Blob Storage (or CDN cache)
    → streams video to browser
```

Output videos are never served directly from api-gateway. The signed Front Door URL is time-limited and CDN-cached, meaning subsequent views of the same output are served from the CDN edge without hitting Blob Storage again.

4. Scalability

Scalability in this platform has two distinct mechanisms: **HTTP concurrency scaling** for request-serving services and **KEDA Service Bus queue-depth scaling** for event-driven workers.

4.1 HTTP concurrency scaling (KEDA HTTP add-on)

Four services scale based on the number of concurrent HTTP requests:

Service	Threshold	Why this number
api-gateway	50 concurrent requests	Balanced for a mix of SSE streams, JSON endpoints, and SAS token generation
mcp-server-analysis	20 concurrent requests	Analysis tool calls are more compute-intensive (YOLO, Whisper, vision)
angular-shell	50 concurrent requests	Static serving; scales on user connection count
librechat	50 concurrent requests	Node.js chat UI; scales on active sessions

When concurrent requests on a service exceed the threshold, ACA adds replicas up to `max_replicas = 20`. When load drops, replicas are removed down to `min_replicas = 1` (in prod, always warm).

4.2 KEDA Service Bus queue-depth scaling

Four services scale based on the number of messages waiting in a Service Bus queue:

Service	Queue monitored	Scale ratio	Rationale
agent-orchestrator	job-queued	1 replica per 5 messages	Each job is CPU/memory intensive; 1:5 prevents over-provisioning
mcp-server-processing	job-queued	1 replica per 5 messages	Processing is memory-heavy (FFmpeg); scales with job volume
preprocessing-worker	video-uploaded	1 replica per 5 messages	FFmpeg keyframe extraction is CPU-heavy
notification-worker	job-completed	1 replica per 5 messages	Email dispatch is lightweight; scale ratio is generous

When a queue is empty, worker services scale to `min_replicas = 1` in production (always one instance ready to pick up new messages immediately). When the queue grows, KEDA calculates `ceil(queue_depth / 5)` replicas up to `max_replicas = 20`.

4.3 Why this architecture scales

- Stateless services** — every application container is stateless. All state (job records, keyframe index, output blobs) is in PostgreSQL and Blob Storage. Adding a replica is always safe.
- Queue-based decoupling** — a burst of video uploads fills the `video-uploaded` queue; the preprocessing-worker scales out to drain it in parallel without any backpressure on the upload path.
- Direct browser uploads** — the api-gateway is never in the video data path for uploads. Users upload gigabyte-scale video files directly to Blob Storage. This means api-gateway replicas are not exhausted by long-running upload connections.
- Keyframe pre-processing** — the agent never sees raw video frames, only 1fps keyframes stored in PostgreSQL. This dramatically reduces the token count sent to Claude, cutting per-job cost and latency regardless of original video length.
- Parallel MCP tool execution** — within a single job, the CrewAI processing agent can invoke multiple MCP tools concurrently where tool dependencies allow. Each tool call is a separate SSE request to a separate mcp-server replica.
- Zone-redundant storage (ZRS)** — Blob Storage data is replicated synchronously across three availability zones. If one zone becomes unavailable, reads and writes continue without interruption.
- Premium Service Bus** — dedicated capacity in the Premium tier ensures consistent message delivery latency under load without resource contention from other tenants.

4.4 Limits and known constraints

- PostgreSQL is a single-replica container.** It cannot scale horizontally. Under very high concurrent job loads, database connection pooling (or PgBouncer) would be needed. This is the primary scalability bottleneck at high concurrency.

- **mcp-server-analysis scales on HTTP concurrency, not queue depth.** Tool calls are synchronous from the agent orchestrator perspective. If tool processing is slow, the agent blocks, which limits throughput per orchestrator replica.
- **GPU workload profiles** — object detection tools (`detect_objects`) use YOLO on CPU in the current container spec. Migrating to an ACA GPU workload profile would be required for sub-second detection on high-resolution keyframes at scale.

5. How New Versions Reach Production

The full pipeline is documented in detail in [gitlab-pipeline.md](#). Here is the production-specific path:

```
Developer pushes to a feature branch
↓
lint (ESLint + Ruff)
↓
build_images — 8 Docker images built and pushed to ACR tagged with commit SHA
↓
aca_test_env_create — ephemeral Azure test environment created by Terraform
↓
deploy_test_services — images pushed to the ephemeral ACA environment
↓
e2e_tests — full end-to-end test suite against live Azure services
  ↓ (always)
collect_e2e_logs — container logs captured as CI artifact before environment is destroyed
  ↓ (always)
aca_test_env_destroy — ephemeral environment fully deleted
  ↓ (main branch only)
push_to_acr — SHA-tagged images re-tagged as :latest
↓
deploy_dev — all 8 container apps in video-extract-dev updated via az containerapp update
↓
manual_approval — human gate in GitLab UI
↓
deploy_prod — all 8 container apps in video-extract-prod updated via az containerapp update
```

Rolling updates: `az containerapp update` with a new image tag triggers a rolling revision in ACA. The old revision stays active and continues serving traffic until the new revision passes its liveness/readiness probes. If the new revision fails to start, ACA automatically routes traffic back to the old revision (no downtime).

Rollback: If a production deployment is bad, a team member goes to GitLab → Deployments → Environments → `prod`, finds the last known-good deployment in the history, and clicks Re-deploy. This re-runs `deploy_prod` with the old image SHA. Alternatively, use the Azure CLI to activate a previous ACA revision directly.

Infrastructure changes (Terraform) to `envs/prod/` are applied manually — not automatically by the pipeline. Only application container images are updated by CI/CD. This is intentional: automated `terraform apply` on persistent environments carries risk of unintended resource modification or destruction.

6. Secret Management in Production

No secrets are hardcoded anywhere in the codebase. The secret chain in production:

```
Developer machine / GitLab CI variables
  ↓ (TF_VAR_* on terraform apply)
Terraform
  ↓
Azure Key Vault          ACA container env vars
(stored for audit,      (injected at deploy time
future managed          via Terraform env{} blocks)
identity wiring)
```

All secrets are stored in GitLab CI as masked variables. They reach the containers at Terraform apply time as container environment variables. Key Vault is also populated with the same values for audit trail and future Phase B managed identity wiring.

In production, `LOCAL_DEV_SKIP_AUTH` is absent (never set), so every API request is validated against Entra External ID. The `OUTPUT_URL_MODE` is `frontdoor`, so all download links are HMAC-signed and time-limited.

7. Observability in Production

All services emit telemetry to a single Application Insights resource (`videoextract-prod-ai`) linked to the shared Log Analytics workspace.

What you get automatically:

- **End-to-end traces** — a user submitting a job creates a trace that spans `api-gateway` → `agent-orchestrator` → `mcp-server-analysis/processing`, with timing for each hop
- **Dependency map** — live topology view of which services call which, with error rates per edge
- **Failure analysis** — exceptions with stack traces, automatically grouped by type
- **Performance** — p50/p95/p99 response times per endpoint; Service Bus processing latency

Where to look for specific things:

Question	Where
Why is a job stuck?	Application Insights → Transaction search → filter by <code>job_id</code> custom property
Which service is slow?	Application Insights → Performance → Operation breakdown
Are Service Bus messages dead-lettering?	Azure portal → Service Bus namespace → Queues → dead-letter count
Container crash / OOM	Log Analytics → Logs query: <code>ContainerAppConsoleLogs_CL</code>
ACA revision health	Azure portal → Container Apps → Revisions → replica provisioning state

There is no Grafana, no custom spans, and no custom metrics. All observability is through the Azure portal Application Insights UI.

8. Production vs Dev vs Test Environment Differences

Aspect	Dev	Prod	Test (ephemeral)
Resource group	<code>video-extract-dev</code>	<code>video-extract-prod</code>	<code>video-extract-test-{pipeline_id}</code>
Lifetime	Persistent	Persistent	Single pipeline run
ACA min replicas	0 (scale to zero)	1 (always warm)	0
ACA max replicas	5	20	3
Service Bus SKU	Standard	Premium	Standard
Blob replication	LRS	ZRS	LRS
PostgreSQL volume	32 GB Azure Files	128 GB Azure Files	32 GB Azure Files
Key Vault purge protection	false	true	Not provisioned
Front Door	Provisioned	Provisioned	Not provisioned
Application Insights	Provisioned	Provisioned	Not provisioned
Auth	Real Entra	Real Entra	<code>LOCAL_DEV_SKIP_AUTH=true</code>
Output URLs	Front Door signed	Front Door signed	Direct Blob URLs
Email	ACS	ACS	ACS